Literature review for ContextKeeper WP-002 Enterprise Delivery Governance

This review consolidates peer-reviewed evidence, vendor data, and practitioner sources across eight research domains supporting WP-002's governance framework for AI-orchestrated software engineering. The findings validate most core claims — particularly the 4x speculative implementation cost ratio, the theoretical basis for multi-model consensus, and the necessity of sequential quality gates — while identifying six significant research gaps where WP-002 makes novel contributions. Across all eight topics, a consistent pattern emerges: AI coding tools amplify existing organizational strengths and weaknesses, making formal governance the critical differentiator between successful and failed AI-assisted development.

Topic 1: The 4x speculative implementation cost ratio finds convergent support

The claim that speculative implementation costs approximately 4x the original to correct is bracketed by multiple independent evidence streams, though no single study directly measures this exact scenario.

Boehm's foundational cost-of-defect curve established the framework. Boehm, B.W. (1981), *Software Engineering Economics*, Prentice-Hall [peer-reviewed], found cost escalation ratios of **1:100 for large defense/government projects** but **approximately 1:4 to 1:5 for small projects**. Boehm and Basili (2001), "Software Defect Reduction Top 10 List," *IEEE Computer* 34(1):135–137, DOI: 10.1109/2.962984 [peer-reviewed], clarified that for "small, non-critical projects" the factor is "more **5:1 than 100:1**" CiteSeerX and that current projects spend **40–50% of effort on avoidable rework**, with **80% of avoidable rework** originating from 20% of defects. PowerShow McConnell, S. (2004), *Code Complete* 2nd Edition, Microsoft Press [practitioner/community evidence], independently reported "more like **1:4** instead of 1:100" for smaller projects. Blogger

The strongest modern empirical validation comes from Menzies, T., Nichols, W., Shull, F., and Layman, L. (2017), "Are Delayed Issues Harder to Resolve? Revisiting Cost-to-Fix of Defects throughout the Lifecycle," *Empirical Software Engineering* 22(4):1903–1935, DOI: 10.1007/s10664-016-9469-x [peer-reviewed]. Across **171 commercial software projects (2006–2014)** — the largest study on this effect — they found no general delayed-issue effect, arXiv ResearchGate but the maximum observed scale-up was **2.16x to 4.37x** for requirements defects detected at integration testing. arXiv This empirical ceiling directly brackets the 4x claim.

Bossavit's critique deserves acknowledgment. Bossavit, L. (2012/2015), *The Leprechauns of Software Engineering*, LeanPub [practitioner/community evidence], traced the Boehm curve's citation history and found the original evidence "isn't up to any reasonable standard of empirical basis." TechWell This critique paradoxically supports the 4x figure as more realistic than the commonly cited exponential curve.

AI-specific evidence strengthens the case considerably. The METR randomized controlled trial — Becker, J., Rush, N., Barnes, E., and Rein, D. (2025), "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity," arXiv:2507.09089 [peer-reviewed preprint] — found that AI tools (Cursor Pro + Claude 3.5/3.7) caused a **19% net slowdown** for 16 experienced developers across 246 tasks, (arXiv) while developers *perceived* they were **20% faster** (METR) — a remarkable 39–44% perception gap. (Codebridge) GitClear's analysis of **211 million lines of code** — Harding, B. and Kloster, M. (2025), "AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones," GitClear [vendor documentation] — found **4× growth in code duplication**, refactoring collapsing from **25% to under 10%** of changed lines, (SoftwareSeni) and code churn rising to **7.9%** (from 5.5% in 2020). (Jonas) CodeRabbit's analysis of **470 GitHub PRs** (2025) [vendor documentation] found AI-generated code contains **1.7× more issues overall**, with performance inefficiencies at ~**8×** and security vulnerabilities at **1.5–2.74×**. (CodeRabbit) The 2024 DORA report [practitioner/community evidence] found (Google Cloud) 25% increased AI adoption correlated with **7.2% decreased delivery stability**. (DevOps Launchpad +3)

⚠️ **Research gap:** No published study directly measures the cost of correcting code written against an *imagined* (speculative) interface versus a *known* interface. The 4x figure is a convergent inference, not a directly measured quantity. Fowler's YAGNI analysis (2015) [practitioner/community evidence] provides the closest conceptual framework, noting that only **⅓ of carefully analyzed features** at Microsoft improved their target metrics (martinfowler) — meaning ⅔ of speculative features are waste.

Topic 2: The Breiman analogy holds but with critical caveats about LLM error correlation

The analogy between Random Forests and multi-model LLM consensus has strong theoretical foundations but faces a significant empirical challenge: frontier LLMs exhibit highly correlated errors.

Foundational ensemble theory. Breiman, L. (2001), "Random Forests," *Machine Learning* 45(1):5–32, DOI: 10.1023/A:1010933404324 [peer-reviewed], established the bound $PE^* \leq \bar{\rho}(1 - s^2)/s^2$, where $\bar{\rho}$ is inter-estimator correlation and s is average strength. (UC Berkeley Statistics) (Semantic Scholar) **Lower correlation between estimators yields lower ensemble error.** Dietterich, T.G. (2000), "Ensemble Methods in Machine Learning," *MCS 2000*, LNCS 1857:1–15, (Typeset) DOI: 10.1007/3-540-45014-9_1 [peer-reviewed], identified three complementary reasons ensembles outperform individuals: the **statistical** reason (insufficient data to identify one best hypothesis), the **computational** reason (local search gets stuck in different optima), and the **representational** reason (the true function may lie outside any single learner's hypothesis space). (Springer +2) Krogh, A. and Vedelsby, J. (1995), "Neural Network Ensembles, Cross Validation, and Active Learning," *NIPS* 7:231–238 [peer-reviewed], proved the Ambiguity

Decomposition: **Ensemble Error = Average Individual Error − Ambiguity**, guaranteeing that more diverse ensembles yield lower error. (Springer)

The correlated-agreement warning. Lakshminarayanan, B., Pritzel, A., and Blundell, C. (2017), "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles," *NeurIPS 2017*, arXiv:1612.01474 [peer-reviewed], showed that when ensemble members trained on similar data agree, this can reflect **shared inductive biases rather than genuine confidence**. (arXiv) (NeurIPS) Deep ensembles with M=5 outperformed MC-Dropout on calibration, and ensemble disagreement increased for out-of-distribution inputs — making disagreement a more reliable signal than agreement. (Campusai)

The critical empirical challenge for LLMs comes from Kim, E., Garg, A., Peng, K., and Garg, N. (2025), "Correlated Errors in Large Language Models," accepted to *ICML 2025*, arXiv:2506.07962 [peer-reviewed]. Evaluating **350+ LLMs**, they found that **models agree 60% of the time when both err**. (arXiv) Crucially, "larger and more accurate models have **highly correlated errors**, even with distinct architectures and providers." (arXiv) This directly challenges the Breiman analogy's premise of error diversity.

Multi-LLM ensemble systems show mixed results. Wang, J. et al. (2024/2025), "Mixture-of-Agents Enhances Large Language Model Capabilities," *ICLR 2025 Spotlight*, arXiv:2406.04692 [peer-reviewed], achieved **65.1% on AlpacaEval 2.0** (vs. 57.5% for GPT-4o) using layered multi-model architectures. (arXiv) (OpenReview) However, Li, W. et al. (2025), "Rethinking Mixture-of-Agents," arXiv:2502.00674 [preprint], found that **Self-MoA** (multiple outputs from one top model) **outperforms** standard MoA (mixing different LLMs) by **6.6%** on AlpacaEval — suggesting intra-model diversity can exceed inter-model diversity when model quality varies substantially. (Hugging Face)

For code generation specifically, Vallecillos Ruiz, F., Hort, M., and Moonen, L. (2025), "Wisdom and Delusion of LLM Ensembles for Code Generation and Repair," arXiv:2510.21513 [preprint], compared 10 LLMs across 3 SE benchmarks and found the ensemble's theoretical ceiling was **83% above the best single model**, but consensus-based selection (majority voting) falls into a **"popularity trap"** that amplifies common incorrect outputs. Diversity-based strategies realized up to **95% of theoretical potential**. (arXiv) EnsLLM (2025), arXiv:2503.15838 [preprint], achieved **90.2% on HumanEval** (vs. 83.5% for GPT-4o alone) using CodeBLEU + behavioral equivalence voting. (arXiv) (arXiv)

Brown, G. (2023), "A Unified Theory of Diversity in Ensemble Learning," *JMLR* 24:1–49 [peer-reviewed], introduced the critical distinction between **"good diversity" and "bad diversity"** (JMLR) (arXiv) — diversity only helps when the ensemble centroid agrees with the true label. (arXiv) This has direct implications for WP-002's consensus protocol design.

⚠️ **Research gaps:** No formal theoretical treatment connecting Breiman's $\bar{\rho}/s^2$ bound to multi-LLM consensus protocols for code generation. No peer-reviewed study applies uncertainty estimation frameworks (à la Lakshminarayanan) to multi-LLM consensus for code. The tension

between Self-MoA (intra-model diversity suffices) and cross-model ensemble theory (inter-model diversity needed) remains unresolved. Hugging Face

---

Topic 3: Governance frameworks improve AI code quality, but controlled evidence is scarce

Strong circumstantial evidence supports the claim that structured governance improves AI-assisted development quality, though no published controlled experiment directly measures governance framework impact.

Benchmark evidence demonstrates scaffolding matters. Jimenez, C.E. et al. (2024), "SWE-bench: Can Language Models Resolve Real-world GitHub Issues?," *ICLR 2024 Oral*, arXiv:2310.06770 [peer-reviewed], GitHub shows that agent-scaffolded systems dramatically outperform raw model baselines. Vals AI Top agents achieve **~80% on SWE-Bench Verified** (Claude Opus 4.5: 80.9%), vellum LLM Leaderboard while raw LLM baselines score far lower. Chen, J. et al. (2026), "SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration," arXiv:2603.03823 [peer-reviewed preprint], found that **75% of tested models break previously working code** during long-term maintenance tasks across **233 days** and **71 consecutive commits** — even when initial patches pass tests. Noqta

Structured prompting yields measurable improvements. Li, J. et al. (2024), "Structured Chain-of-Thought Prompting for Code Generation," *ACM TOSEM*, DOI: 10.1145/3690635 [peer-reviewed], found SCoT prompting outperformed standard CoT by up to **13.79% Pass@1 on HumanEval** OpenReview and **12.31% on MBPP**. arXiv Cruz, H. et al. (2025), "Prompt Engineering and Framework: Implementation to Increase Code Reliability," arXiv:2506.10989 [preprint], showed the ADIHQ template achieves **>1.5× higher test-pass-per-token ratio** than Chain-of-Thought. Emergent Mind

Specification completeness evidence is emerging but incomplete. The VERINA benchmark (2025), arXiv:2505.23135 [preprint], found that even top-performing LLMs achieve only **61.4% correct code**, **51.0% sound specifications**, and **3.6% successful proof** arXiv — with manually enhanced specifications significantly improving results. The SBC metric paper (2025), arXiv:2502.07835 [preprint], demonstrated that reverse-generating requirements from AI code can quantify specification–implementation gaps and identify hallucinations. arXiv

Multiple failure taxonomies have been published, including an 8-category taxonomy from a 72-study survey (2025, arXiv:2512.05239 [preprint]), arXiv a 3-level hierarchy from Song, Z. et al. (2023), *MAPS workshop at FSE* [peer-reviewed], Mapsworkshop and the AI-ODC framework (2025, arXiv:2508.17900 [preprint]) extending Orthogonal Defect Classification with a novel AI attribute. arXiv CodeRabbit (2025) [vendor documentation] found AI-generated code makes more mistakes "when it lacks business rules, configuration patterns, or architectural constraints" CodeRabbit — directly supporting specification-completeness governance. CodeRabbit

DORA 2025 findings are foundational. The 2025 DORA report (~5,000 respondents) [practitioner/community evidence] TechTarget found **~90% AI adoption**, Google with AI acting as an "amplifier" — magnifying both strengths and dysfunctions. Efficientlyconnected Google They added **Rework Rate as a 5th core metric** Google Cloud DX and introduced 7 team archetypes. Efficientlyconnected Faros AI telemetry (>10,000 developers) [vendor documentation] documented the "AI Productivity Paradox": **21% more tasks completed, 98% more PRs merged**, Medium but organizational delivery metrics stay flat. Faros

⚠️ **Research gaps:** No peer-reviewed controlled study measures governance framework impact on AI development quality. No study quantifies minimum specification completeness thresholds. No failure taxonomy classifies failures by governed vs. ungoverned conditions. The CURRANTE registered report (Rosa, G. et al., SANER 2026 [peer-reviewed]) will test specification-driven code generation with 32 professional developers, but results are pending. CodeNote

---

Topic 4: Model routing achieves 50–98% cost reduction but lacks SE-specific validation

Strong evidence supports task-to-model routing as a viable optimization strategy, with multiple peer-reviewed systems demonstrating dramatic cost savings on general NLP benchmarks — though SE-task-type-specific routing remains an open research area.

Cross-benchmark performance divergence provides direct evidence for routing. Aggregating leaderboard data reveals substantial model specialization: Claude Opus 4.6 achieves **80.8% on SWE-Bench** but **76% on LiveCodeBench**, while OpenAI o3-mini scores **74.1% on LiveCodeBench** but only **61% on SWE-Bench**. GPT-4.5 leads in tool use (BFCL: **69.9%**) despite mediocre SWE-Bench performance (38%). vellum Price differentials span **100×** between frontier ($5/$25 per M tokens for Claude Opus) and budget models Iclr ($0.08/$0.35 for GPT oss 20b). vellum

Foundational routing frameworks. Chen, L., Zaharia, M., and Zou, J. (2023), "FrugalGPT," arXiv:2305.05176 [peer-reviewed], demonstrated **up to 98% cost reduction** while matching GPT-4 performance, arXiv arXiv finding that a cheap model produces identical outputs on **80%+ of queries**. OpenReview Ong, I. et al. (2025), "RouteLLM: Learning to Route LLMs with Preference Data," *ICLR 2025* [peer-reviewed], achieved **>85% cost reduction on MT-Bench** and **>2× cost reduction** on widely-recognized benchmarks, with routers that generalize to unseen model pairs. LMSYS Org OpenReview Ding, D. et al. (2024), "Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing," *ICLR 2024* [peer-reviewed], reduced large-model calls by **up to 40%** with no quality drop. OpenReview Aggarwal, P. et al. (2024), "AutoMix: Automatically Mixing Language Models," *NeurIPS 2024* [peer-reviewed], achieved **>50% cost reduction** arXiv with a 3-class router including an "abstain" class for unsolvable tasks.

Code-specific routing is emerging. Cheng, J. et al. (2025), "AdaptiveLLM: A Framework for Selecting Optimal Cost-Efficient LLM for Code-Generation Based on CoT Length," arXiv:2506.10525 [preprint], proposes difficulty-based routing using Chain-of-Thought length clustering. (arXiv) RouterBench — Hu, Q.J. et al. (2024), *ICML 2024 Workshop*, arXiv:2403.12031 [peer-reviewed] — established with 405,467 inference outcomes that **no single model consistently outperforms all others** across tasks. (arXiv)

Commercial model registries exist. Azure AI Foundry's Model Catalog indexes **1,900+ models** with filterable capabilities and built-in evaluation [vendor documentation]. (Microsoft Learn) The open-source llm-registry (GitHub) supports **10,000+ models** with normalized benchmark scores and capability filtering [practitioner/community evidence]. (GitHub) CDT Framework (2025), arXiv:2509.24422 [preprint], provides theoretical capability classification with **93.1% tagging accuracy**. (arXiv)

⚠️ **Research gaps:** No SE-task-type-specific routing benchmark exists across architecture design, debugging, documentation, code generation, testing, and refactoring. Cost-quality routing has not been validated specifically for SE tasks. No longitudinal studies track whether routing decisions remain optimal across model updates. Architecture-level tasks are essentially unbenchmarked.

Topic 5: LLM-generated invariants reach 72% coverage, but contract vs. testing comparison is unmeasured

Formal invariant enforcement for AI-generated code is advancing rapidly, with LLMs now capable of generating verifiable specifications for the majority of programs, though the empirical comparison of contracts versus testing alone remains a genuine research gap.

Design by Contract foundations. Meyer, B. (1992), "Applying 'Design by Contract'," *IEEE Computer* 25(10):40–51, DOI: 10.1109/2.161279 [peer-reviewed], established contracts (preconditions, postconditions, invariants) as executable specifications. (Wikipedia) Kudrjavets, G., Nagappan, N., and Ball, T. (2006), "Assessing the Relationship between Software Assertions and Faults," *ISSRE 2006*:204–212, DOI: 10.1109/ISSRE.2006.19 [peer-reviewed], found **statistically significant negative correlation** between assertion density and fault density in Microsoft Visual Studio, with assertions more effective than static analysis tools. (Microsoft +2) Kochhar, P.S. and Lo, D. (2017), *EASE 2017*:298–307 [peer-reviewed], replicated this in **185 Apache Java projects**.

LLM specification generation has reached practical viability. Ma, L. et al. (2025), "SpecGen: Automated Generation of Formal Program Specifications via Large Language Models," *ICSE 2025*, DOI: 10.1109/ICSE55347.2025.00129 [peer-reviewed], generated verifiable JML specifications for **279 out of 385 programs (72.5%)**. (arXiv) Sun, C. et al. (2025), "ClassInvGen" [peer-reviewed], improved class invariant correctness from **77% to 100%** through invariant-test co-generation. (OpenReview) Wu, H. et al. (2024), "LaM4Inv," *ASE 2024*:1395–1407 [peer-

reviewed], generated **309 correct loop invariants out of 316 benchmark problems.** Chentaolue Richter, C. et al. (2025), "NL2Contract," arXiv:2510.12702 [preprint], demonstrated translating natural-language docstrings into executable Python contracts, with generated contracts reducing false alarms compared to postconditions alone. Endres, M. et al. (2024), *FSE 2024* [peer-reviewed], showed LLM-generated postconditions caught **64 real-world historical bugs from Defects4J.** arXiv

Regulatory frameworks create compliance pull. EU AI Act Article 9 (Regulation 2024/1689) EU Artificial Intelligence Act [regulatory] requires a **continuous iterative risk management system** throughout the AI lifecycle, Label Studio Europa including testing against "prior defined metrics and probabilistic thresholds" Euaiact Artificial-intelligence-act (Art. 9(8)). NIST SP 800-53 Rev. 5 [government standard] provides controls **SI-7** (Software/Firmware Integrity verification), **SA-11** (Developer Testing and Evaluation), and **SA-15** (Development Process, Standards, and Tools). Accorian NIST is developing COSAiS (Control Overlays for Securing AI Systems, concept paper August 2025) [vendor documentation, in development] specifically mapping SP 800-53 controls to AI use cases. NIST SP 800-218A (2024) [government standard] establishes secure development practices for generative AI. NIST CSRC

⚠️ **Research gaps:** No controlled experiment directly compares DbC + testing versus testing-only defect detection rates. Current LLM invariant benchmarks are limited to small programs (~53 LOC average). arXiv No published analysis connects EU AI Act Article 9 specifically to formal invariant enforcement as a compliance mechanism. COSAiS overlays remain in development.

Topic 6: GPT-4 code executability dropped from 52% to 10% in three months

System drift is empirically documented as a severe risk for AI-assisted development, with the most dramatic evidence coming from direct measurements of LLM behavioral change over time.

The landmark LLM drift study. Chen, L., Zaharia, M., and Zou, J. (2023), "How Is ChatGPT's Behavior Changing over Time?," *Harvard Data Science Review* [peer-reviewed], found GPT-4's directly executable code generation dropped from **52.0% to 10.0% between March and June 2023** OpenReview — a catastrophic degradation in 3 months. GPT-3.5 code executability dropped from **22.0% to 2.0%** OpenReview over the same period. GPT-4 prime number identification accuracy fell from **84% to 51%.** Semantic Scholar These findings demonstrate that LLM API services undergo substantial behavioral drift on timescales incompatible with any assumption of stable behavior.

Concept drift theory provides the detection framework. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014), "A Survey on Concept Drift Adaptation," *ACM Computing Surveys* 46(4):Article 44, DOI: 10.1145/2523813 [peer-reviewed], provides the canonical taxonomy Semantic Scholar (sudden, gradual, incremental, recurring drift) and detection

algorithms (DDM, ADWIN). Sculley, D. et al. (2015), "Hidden Technical Debt in Machine Learning Systems," *NeurIPS* 28:2503–2511 [peer-reviewed], identified ML-specific risks including boundary erosion, the **"CACE" principle** (Changing Anything Changes Everything), and hidden feedback loops. ResearchGate +2

Architectural drift from AI-generated code is emerging as a measurable phenomenon. Li, R. et al. (2022), "Understanding Software Architecture Erosion," *J. Software: Evolution and Process* 34(3):e2423, DOI: 10.1002/smr.2423 [peer-reviewed], cataloged erosion symptoms across **73 studies** into structural, violation, quality, and evolution categories. ResearchGate ResearchGate A 2025 survey of 40 practitioners (arXiv:2506.17833 [preprint]) found that **AI-generated code for larger problems shows signs of architectural erosion, primarily lower cohesion**. arXiv GitClear's analysis [vendor documentation] quantified this: an **8-fold increase in duplicated code blocks** Jonas and refactoring collapsing from 25% to under 10% of changes. Sonar +3

Observability standards are developing but immature. The OpenTelemetry GenAI Semantic Conventions [vendor documentation, CNCF project] define span attributes for AI operations (`gen_ai.operation.name`, `gen_ai.request.model`, token usage metrics) with provider-specific extensions for OpenAI, Anthropic, and AWS Bedrock — but these remain in **"Development" status**. OpenTelemetry OpenLLMetry (Traceloop) [practitioner/community evidence] provides the most mature open-source LLM observability library with auto-instrumentation across major providers. Dynatrace GitHub LangSmith [vendor documentation] represents commercial state-of-the-art with trace clustering for pattern and anomaly detection. LangChain

⚠️ **Research gaps:** No peer-reviewed work specifically studies LLM behavioral drift in AI-orchestrated SE pipelines. No tools or methods are calibrated for detecting AI-specific architectural drift patterns. No formal drift detection algorithms exist for OpenTelemetry GenAI telemetry streams. The intersection of LLM drift × code quality × pipeline governance is entirely unstudied.

Topic 7: Temperature=0 produces 30/30 unique GPT-4 outputs — reproducibility requires architectural solutions

LLM output non-determinism at temperature=0 is now well-documented, with root causes identified and solutions emerging — but no standard addresses AI-generated code reproducibility.

Temperature=0 does not produce deterministic outputs. Atil, B. et al. (2025), "Non-Determinism of 'Deterministic' LLM Settings," arXiv:2408.04667v5 [peer-reviewed preprint], found accuracy variations up to **15%** across 10 runs at temperature=0 for 5 LLMs, with best-to-worst performance gaps up to **70%**. arXiv arXiv Chann, S. (2023) [practitioner/community evidence] ran 30 identical prompts and found GPT-4 produced **30/30 unique completions** at temperature=0 152334H with max_tokens=256, while a non-MoE base model (davinci) was

deterministic (1/30 unique). (github) All three major providers explicitly disclaim determinism: OpenAI describes seed-based reproducibility as "mostly deterministic" (Unstract) [vendor documentation], Anthropic states "even with temperature of 0.0, the results will not be fully deterministic" (Substack) [vendor documentation], and Google notes "a small amount of variation is still possible" (Substack) [vendor documentation].

The root cause is batch-size-dependent computation, not floating-point randomness. He, H. (Thinking Machines Lab, 2025), "Defeating Nondeterminism in LLM Inference," DOI: 10.64434/tml.20250910 [practitioner/community evidence], demonstrated that testing Qwen3-235B-A22B at temperature=0 with 1,000 identical prompts yielded **80 unique completions** under standard inference, but **1,000/1,000 identical completions** with batch-invariant kernels. (Thinkingmachines) The true culprit is not MoE routing per se but **non-batch-invariant GPU kernels** in RMSNorm, matrix multiplication, and attention operations — batch size varies with server load, producing per-user non-determinism. (Bartosz Mikulski +2) LMSYS (2025) [practitioner/community evidence] integrated batch-invariant kernels into SGLang at ~**34.35% throughput overhead**, achieving 100% reproducible outputs. (LMSYS Org) (Keywordsai)

Regulated industries require reproducibility but lack AI-specific guidance. Federal Reserve SR 11-7 (2011) (Federal Reserve) [regulatory] requires independent validation and outcome analysis for models, (Krista AI) creating implicit reproducibility obligations. The FDA's Good Machine Learning Practice principles (2021) [regulatory] explicitly require making training "reproducible and auditable." (Rookqs) The EU AI Act (2024) (European Commission) [regulatory] requires reproducibility for GPAI model evaluations (Code of Practice), comprehensive audit trails, (Artificial-intelligence-act) and **10-year documentation retention.** (EU Artificial Intelligence Act) The Carnegie Endowment (2024) [policy research] assessed 8 AI standards under the EU AI Act and found **all had "middling or poor" maturity.** (Carnegie Endowment for In...)

⚠️ **Research gaps:** No ISO, IEEE, NIST, or other standard specifically addresses AI-generated code output reproducibility. No framework defines "acceptable non-determinism" thresholds for different risk contexts. No published work maps regulatory requirements to specific technical mitigations for LLM non-determinism. Batch-invariant inference is not available in any commercial API. No academic work proposes "reproducibility contracts" for multi-agent AI code generation — this is a novel contribution opportunity for WP-002.

Topic 8: Testing alone caps at ~85% DRE — multiple sequential gates are required for >95%

The research literature strongly supports sequential quality gates as the only known method to achieve high defect removal efficiency, with foundational evidence from inspections, cost escalation studies, and phase containment metrics.

Fagan's inspection research established the benchmark. Fagan, M.E. (1976), "Design and Code Inspections to Reduce Errors in Program Development," *IBM Systems Journal* 15(3):182–211 [peer-reviewed], found design and code inspections located **82% of all errors**, (IfSQ) with **38 defects per KNCSS** found via inspections versus 8 via unit testing. (Flylib) Inspections improved productivity by **20%+**. (IfSQ) Kolkhorst, B.G. and Macina, A.J. (1988), *IEEE AES Magazine*:25–31 [peer-reviewed], showed IBM Houston's shuttle software reduced product defect rates by **~70%** as early detection percentages increased from ~50% to >85%. (Flylib) Briand, L.C. et al. (1998) [peer-reviewed] found code inspections reduce lifecycle defect detection costs by **39%** and design inspections by **44%**. (ScienceDirect)

Capers Jones provides the definitive case for multiple complementary gates. Jones, C. and Bonsignour, O. (2011), *The Economics of Software Quality*, Addison-Wesley [practitioner/community evidence], (Amazon) found U.S. average defect removal efficiency (DRE) is approximately **85%**, with best-in-class projects achieving **>99%**. (Academia.edu) Testing alone has an efficiency ceiling of **~35% per stage**. Formal inspections average **~85% DRE** and static analysis **~55% DRE**. (ResearchGate) The critical finding: **synergistic combination of pre-test inspections + static analysis + formal testing is the only known way to achieve cumulative DRE >95%**. (TestDino) (PPI) Only 30–35% of defects originate in code; 20–25% from requirements, 25–30% from design.

Cost escalation data justifies catching defects early. The NASA Johnson Space Center (2010), "Error Cost Escalation Through the Project Life Cycle," Technical Report 20100036670 [peer-reviewed], compiled cost factors across spacecraft, aircraft (231 errors), and satellite systems. (nasa) Software median factors: **Requirements=1×, Design=5×, Code=10×, Test=50.5×**. (nasa) NIST/RTI (2002), "The Economic Impacts of Inadequate Infrastructure for Software Testing," NIST Planning Report 02-3 [government report], estimated software bugs cost **$59.5 billion annually** (0.6% (Abeacha) of GDP), with **>$22 billion** eliminable through better testing infrastructure.

Phase containment effectiveness provides the measurement framework. Hevner, A.R. (1997), "Phase containment metrics for software quality improvement," *Information and Software Technology* 39(13):867–877 [peer-reviewed], formalized PCE = Errors/(Errors + Defects), where errors are found in-phase and defects escape. Industry Total Containment Effectiveness ranges from **65% to 98%**, with typical organizations at **75–85%** (Hallowell, iSixSigma [practitioner/community evidence]). Cooper, R.G. (1990/2008), *J. Product Innovation Management* [peer-reviewed], found **88% of U.S. firms** in new product development apply Stage-Gate® processes.

AI-specific quality gate evidence is emerging. The 2025 DORA report [practitioner/community evidence] found **9% increase in bug rates** correlated with AI adoption, making quality gates essential. Multi-agent coding surveys (2025, arXiv:2508.00083 [preprint]) document converging pipeline architectures (AgentCoder, CodePori, MAGIS) with implicit sequential gates —

programmer → test designer → test executor — but no formal quality gate protocol has been proposed.

⚠️ **Research gaps:** No formal "Quality Gate Protocol" for AI coding agents exists in published literature. No study measures PCE specifically for AI-generated code. All cost escalation data predates AI-assisted development. No empirical comparison of different gate configurations (2-gate vs. 4-gate vs. 6-gate) for AI pipelines. The optimal number and ordering of gates for AI-orchestrated development has not been empirically established — WP-002's 4-gate protocol (Build/Proof/Operations/Architecture) is a novel contribution.

Conclusion: six novel contributions emerge from the gaps

This literature review reveals that WP-002's core claims find **convergent support across multiple independent evidence streams**, while simultaneously identifying **six areas where the paper makes genuinely novel contributions** to the field:

1. **Speculative implementation cost quantification** — the 4× figure is bracketed by Boehm (1:4), Menzies (2.16–4.37×), and GitClear (4× duplication growth), but no study directly measures the speculative-vs-known-interface cost differential
2. **Formal multi-model consensus protocol** — while Kim et al. (ICML 2025) empirically demonstrates 60% error correlation and Vallecillos Ruiz identifies the "popularity trap," no governance-oriented consensus protocol with structured disagreement taxonomy exists
3. **Sequential quality gate protocol for AI agents** — Jones' evidence that >95% DRE requires multiple complementary techniques is well-established, but no formal gate protocol has been proposed for AI-orchestrated development
4. **Reproducibility contracts** — Thinking Machines (2025) proved batch-invariant determinism is achievable at 34% overhead, but no academic work defines reproducibility contracts for multi-agent AI code generation
5. **AI-specific drift monitoring** — Chen et al.'s 52%→10% code executability drop provides the motivation, but no system integrates LLM behavioral monitoring with code quality and architectural conformance checking
6. **Formal invariant enforcement for AI-generated code** — SpecGen achieves 72.5% verifiable specification generation and LaM4Inv generates 309/316 correct invariants, but no end-to-end governance pipeline connects LLM code generation → contract generation → runtime verification → compliance reporting

The strongest external validation comes from the **2025 DORA report's "amplifier" finding**: AI magnifies existing organizational strengths and weaknesses, making governance the

determining factor for AI-assisted development success. WP-002's governance framework directly addresses this amplification dynamic by imposing formal structure where the evidence shows informal practices fail.